

Finance Dashboard API

Backend Developer Assignment Submission

Submitted By: Mohd Saqib

Email: saqibansari2011@gmail.com

GitHub Repository: [imsaqib04](#)

LinkedIn: [imsaqib04](#)

Date: April 06, 2026

Finance Data Processing and Access Control

1. Project Overview

This project is a backend system for a Finance Dashboard where different users interact with financial records based on their assigned roles. The system supports user management, financial record management, role-based access control, and dashboard-level analytics.

Tech Stack:

- Backend Framework: Django + Django REST Framework
 - Database: MYSQL
 - Authentication: JWT (JSON Web Tokens)
 - Additional Libraries: django-filter, djangorestframework-simplejwt
-

2. Roles and Permissions

API	Viewer	Analyst	Admin
Register	✓	✓	✓
Login	✓	✓	✓
View Dashboard Summary	✓	✓	✓
View Records	✗	✓	✓
Create Record	✗	✗	✓
Update Record	✗	✗	✓
Delete Record	✗	✗	✓
Update User Role	✗	✗	✓
View Deleted Records	✗	✓	✓

3.Setup Instructions

Step 1 — Clone the repository

```
git clone https://github.com/imsaqib04/Finance_dashboard
cd Finance_dashboard
```

Step 2 — Create and activate Virtual Environment

Windows:

```
python -m venv venv
```

```
venv\Scripts\activate
```

Mac / Linux:

```
python -m venv venv
```

```
source venv/bin/activate
```

Step 3 — Configure MySQL Database

Create database in MySQL:

```
CREATE DATABASE finance_db;
```

Configure settings.py:

```
DATABASES = {  
  
'default': {  
  
    'ENGINE': 'django.db.backends.mysql',  
  
    'NAME': 'finance_db',  
  
    'USER': 'your_mysql_username',  
  
    'PASSWORD': 'your_mysql_password',  
  
    'HOST': 'localhost',  
  
    'PORT': '3306',  
  
}  
  
}
```

Step 4 — Install dependencies

```
pip install -r requirements.txt
```

Step 5 — Run migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 6 — Create Admin user

```
python manage.py createsuperuser
```

Step 7 — Set Admin role in shell

```
python manage.py shell

from users.models import User

user = User.objects.get(username='your_username')

user.role = User.Role.ADMIN

user.save()

exit()
```

Step 8 — Run server

```
python manage.py runserver
```

Step 9 — Access the API

API Base URL: <http://127.0.0.1:8000/>

Django Admin Panel: <http://127.0.0.1:8000/admin/>

4. API Documentation

API 1: User Registration

Field	Details
Method	POST
URL	/api/auth/register/
Permission	Public
Auth Required	No

Description: Registers a new user in the system. The default role assigned is VIEWER. Both username and email must be unique — this is enforced for security and data integrity purposes. Duplicate username or email returns a 400 error with a clear message.

Request Body:

```
{
  "first_name": "mohammad",
  "last_name": "saqib",
  "username": "iamsaqibansari",
  "email": "saqibtest@gmail.com",
```

```
"password": "saqib@1234"
```

```
}
```

Success Response — 201 Created:

```
{
```

```
  "success": true,
```

```
  "message": "User registered successfully!",
```

```
  "data": {
```

```
    "username": "iamsaqibansari",
```

```
    "email": "saqibtest@gmail.com"
```

```
  }
```

```
}
```

```
}
```

Error Response — 400 Bad Request:

```
{
```

```
  "success": false,
```

```
  "message": "Registration failed",
```

```
  "errors": {
```

```
    "username": ["A user with that username already exists."],
```

```
    "email": ["This email is already registered."]
```

```
}
```

}

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:8000/api/auth/register/
- Body:** raw (selected), containing a JSON object:

```
{  "first_name": "mohammad",  "last_name": "saqib",  "username": "iamsaqibansari",  "email": "saqibtest@gmail.com",  "password": "saqib1234"}
```
- Response:** 201 Created, 1.09 s, 441 B. The JSON body is:

```
{  "success": true,  "message": "User registered successfully!",  "data": {    "username": "iamsaqibansari",    "email": "saqibtest@gmail.com"  }}
```

Figure 1: User Registration — Success (201 Created)

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:8000/api/auth/register/
- Body:** raw (selected), containing a JSON object:

```
{  "first_name": "mohammad",  "last_name": "saqib",  "username": "iamsaqibansari",  "email": "saqibtest@gmail.com",  "password": "saqib1234"}
```
- Response:** 400 Bad Request, 72 ms, 483 B. The JSON body is:

```
{  "success": false,  "message": "Registration failed",  "errors": {    "username": [      "A user with that username already exists."    ],    "email": [      "This email is already registered."    ]  }}
```

Figure 2: User Registration — Duplicate Error (400)

API 2: User Login

Field	Details
Method	POST
URL	/api/auth/login/
Permission	Public
Auth Required	No

Description: Authenticates the user and returns a JWT access token and refresh token. The access token must be included in the Authorization header for all protected API requests.

Request Body:

```
{
  "username": "admin",
  "password": "Admin@1234"
}
```

Success Response — 200 OK:

```
{
  "access": "eyJhbGciOiJI...",
  "refresh": "eyJhbGciOiJI..." }
```

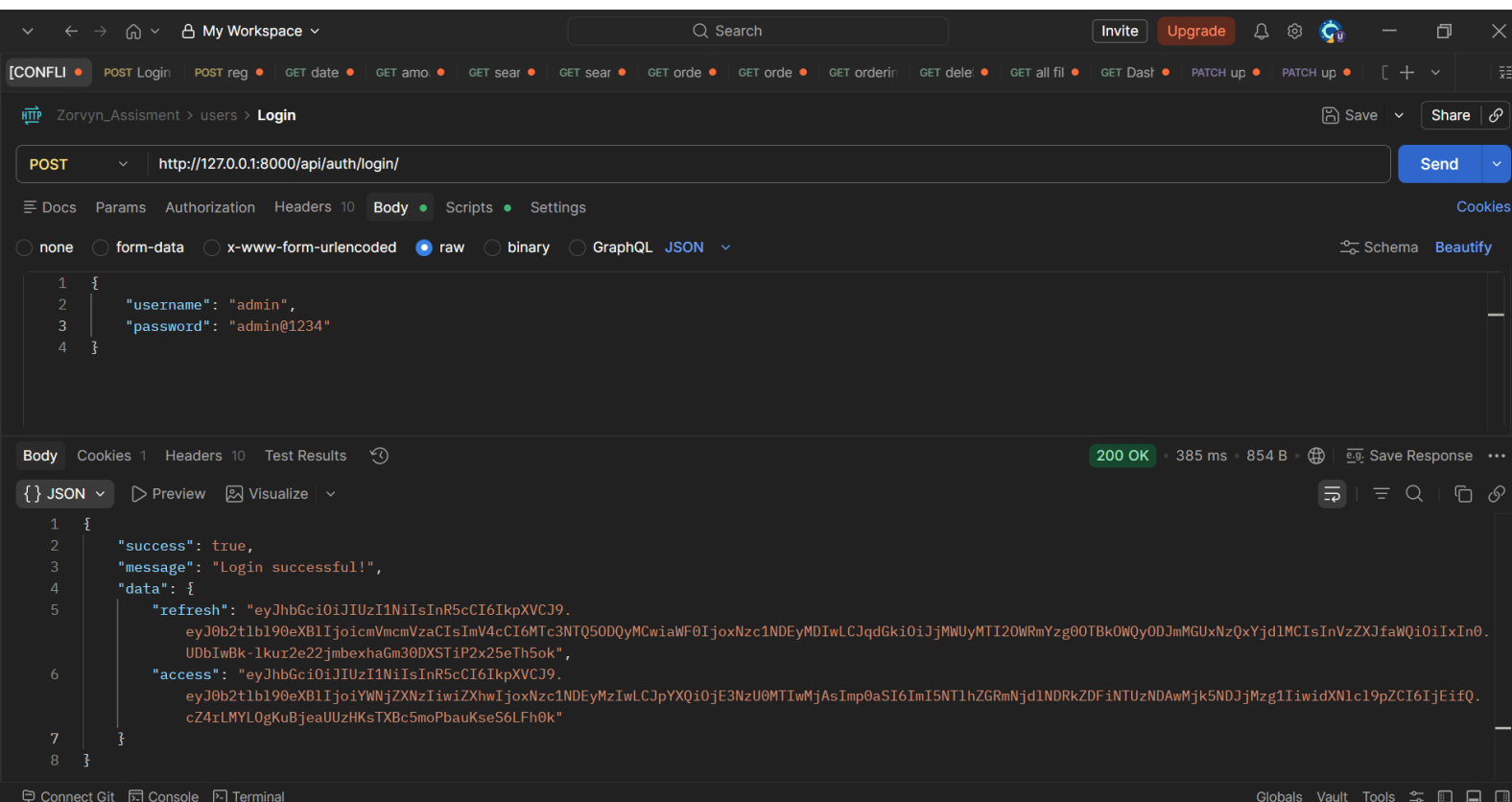


Figure 3: User Login — Success (200 OK)

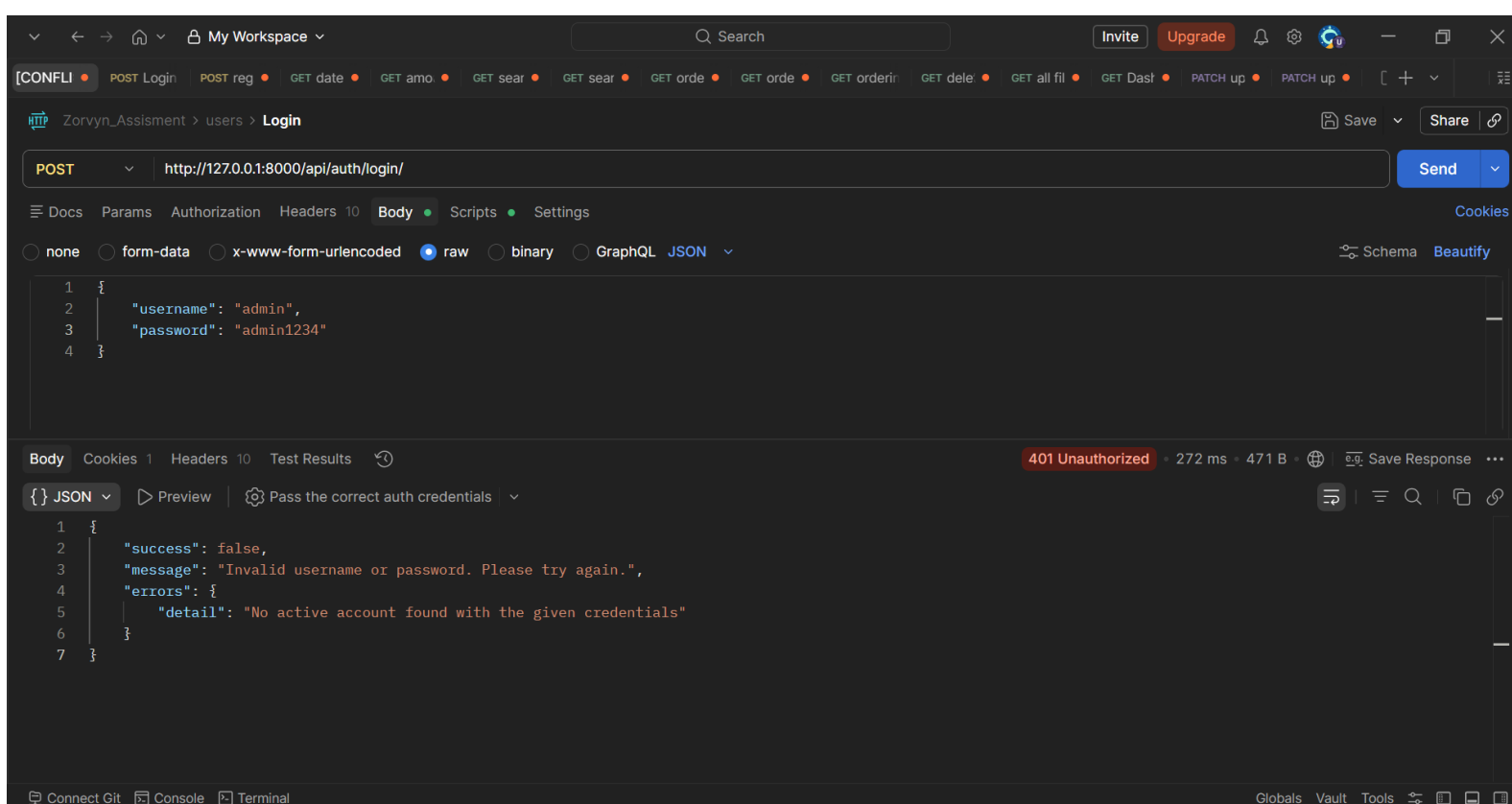


Figure 4: User Login - Unauthorized (401)

API 3: Update User Role (Admin Only)

Field	Details
-------	---------

Method	PATCH
--------	-------

URL	/api/users/{id}/update/
-----	-------------------------

Permission	Admin Only
------------	------------

Auth Required Yes

Description: Allows Admin to update the role or active status of any user. A regular user cannot change their own role — this prevents privilege escalation attacks.

Request Body:

```
{  "role": "ANALYST",  "is_active": true}
```

Success Response — 200 OK:


```
{  
  "success": true,  
  "message": "User ID 6 role/status updated successfully!",  
  "data": {  
    "role": "ANALYST",  
    "is_active": true  
  }  
}
```

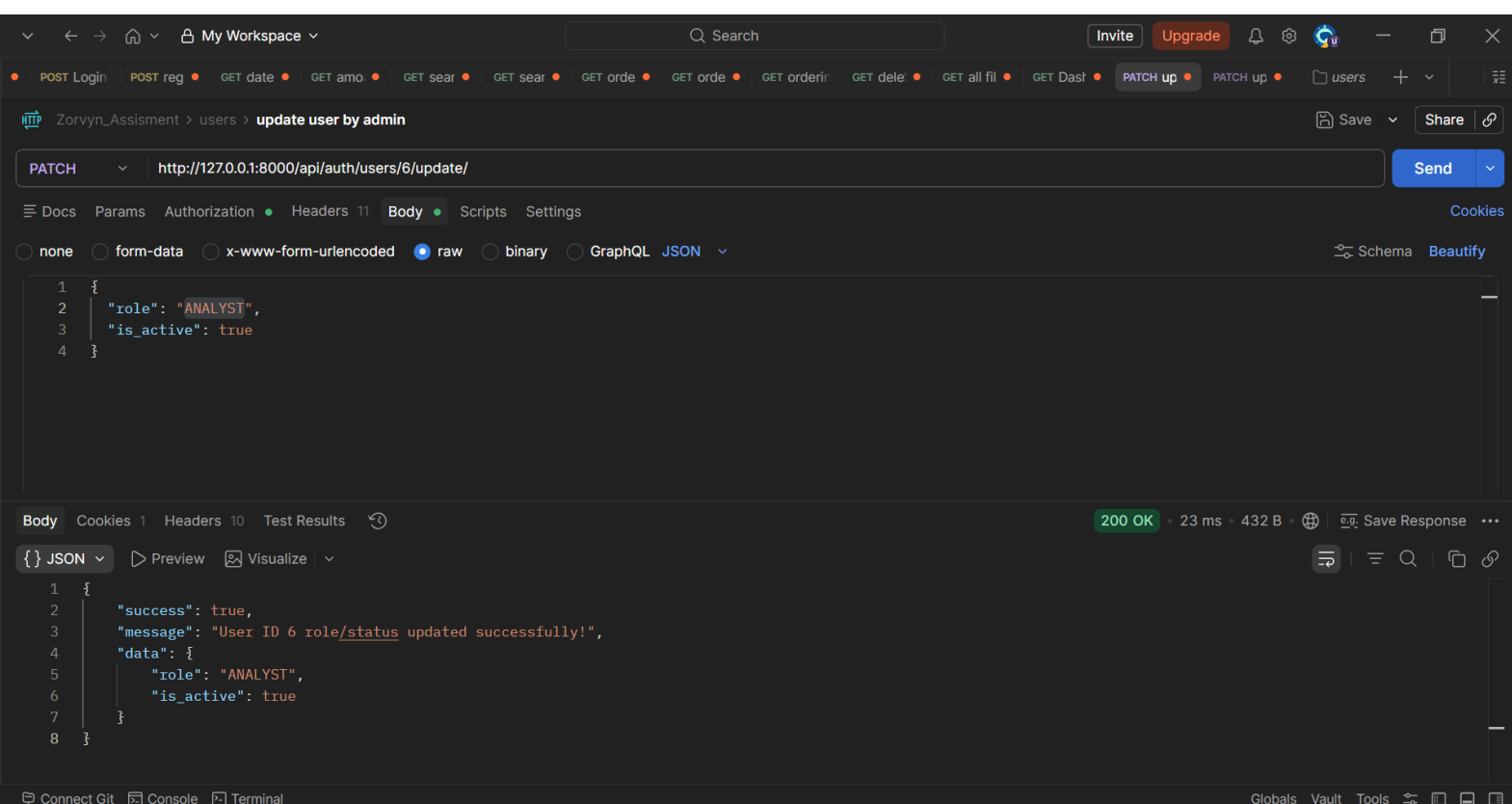


Figure 5: Update User Role – Admin only (200 OK)

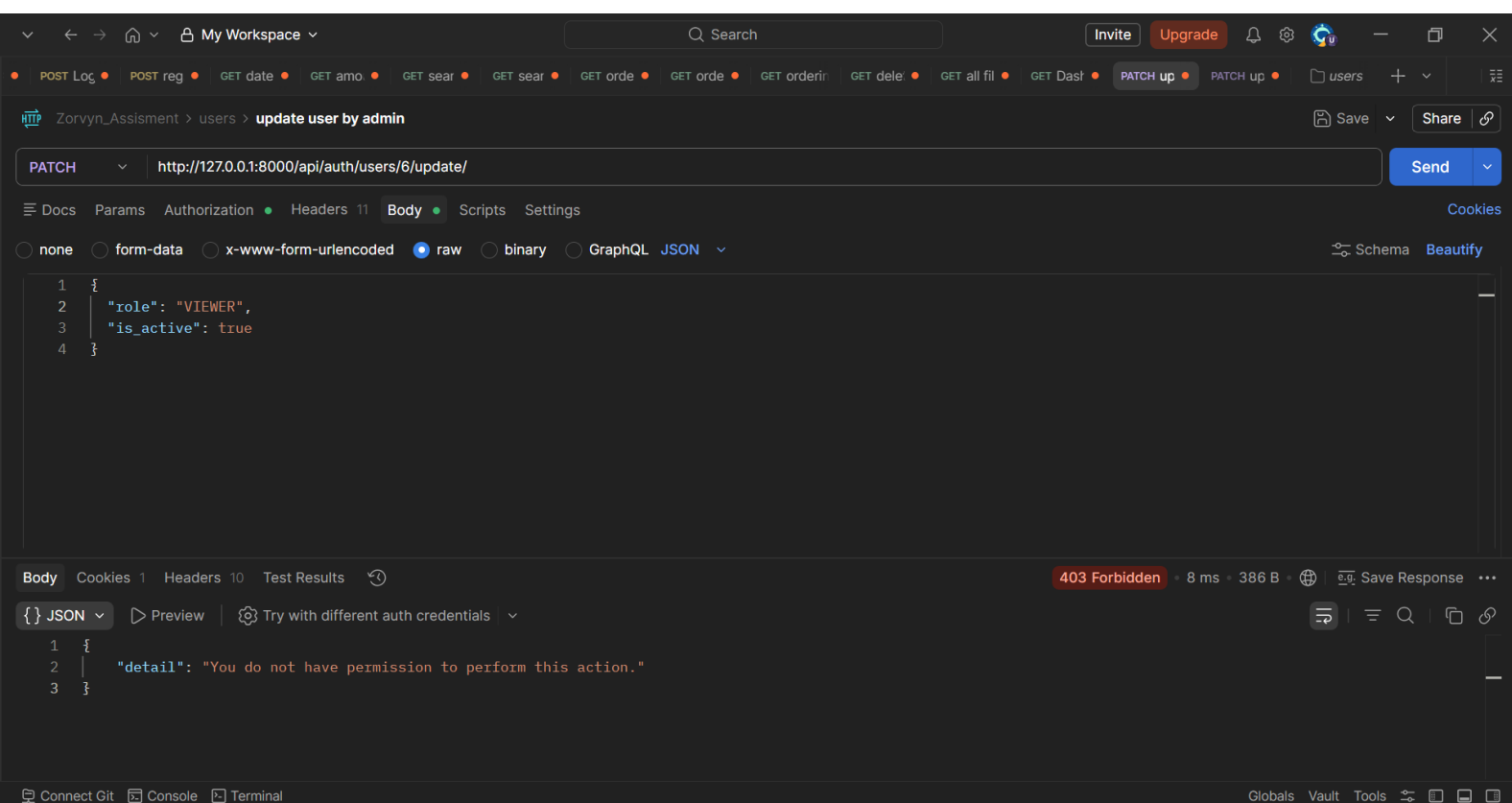


Figure 6: Update User Role – Viewer/Analyst (403 Forbidden)

API 4: Create Financial Record (Admin Only)

Field	Details
Method	POST
URL	/api/records/
Permission	Admin Only
Auth Required	Yes

Description: Creates a new financial record. The record is automatically linked to the logged-in Admin user via the created_by field. Amount must be greater than zero.

Request Body:

```
{  "amount": "55000.00",  "record_type": "INCOME",  "category": "Salary",  "date": "2024-01-15",  "notes": "January monthly salary"}
```

```
}
```

Success Response — 201 Created:

```
{
```

```
  "success": true,
```

```
  "message": "Financial record added successfully!",
```

```
  "data": {
```

```
    "id": 21,
```

```
    "amount": "55000.00",
```

```
    "record_type": "INCOME",
```

```
    "category": "Salary",
```

```
    "date": "2024-01-15",
```

```
    "notes": "January monthly salary",
```

```
    "is_deleted": false,
```

```
    "created_by": 1
```

```
  }
```

```
}
```

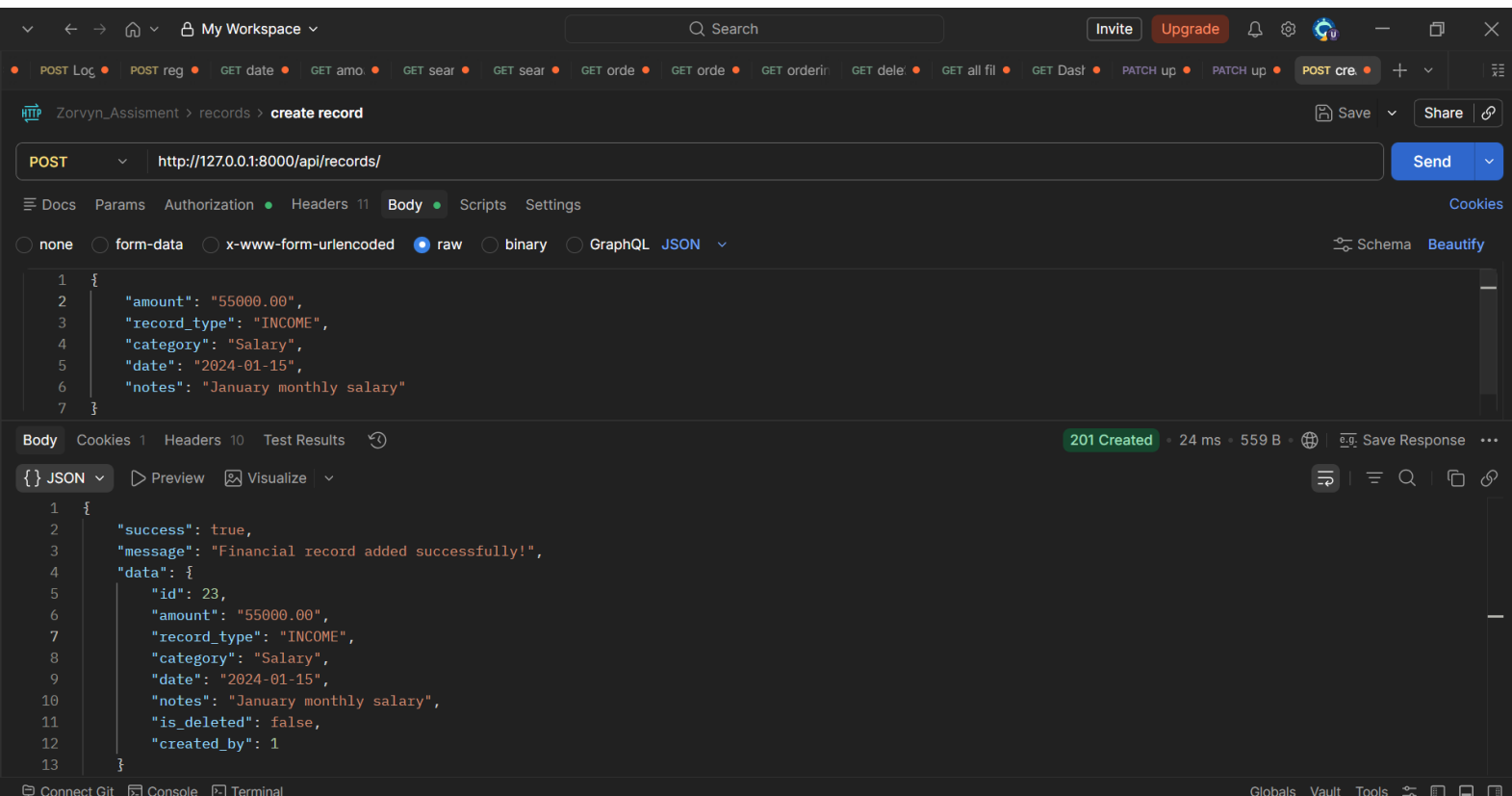


Figure 7: Create Financial Record – Success (201)

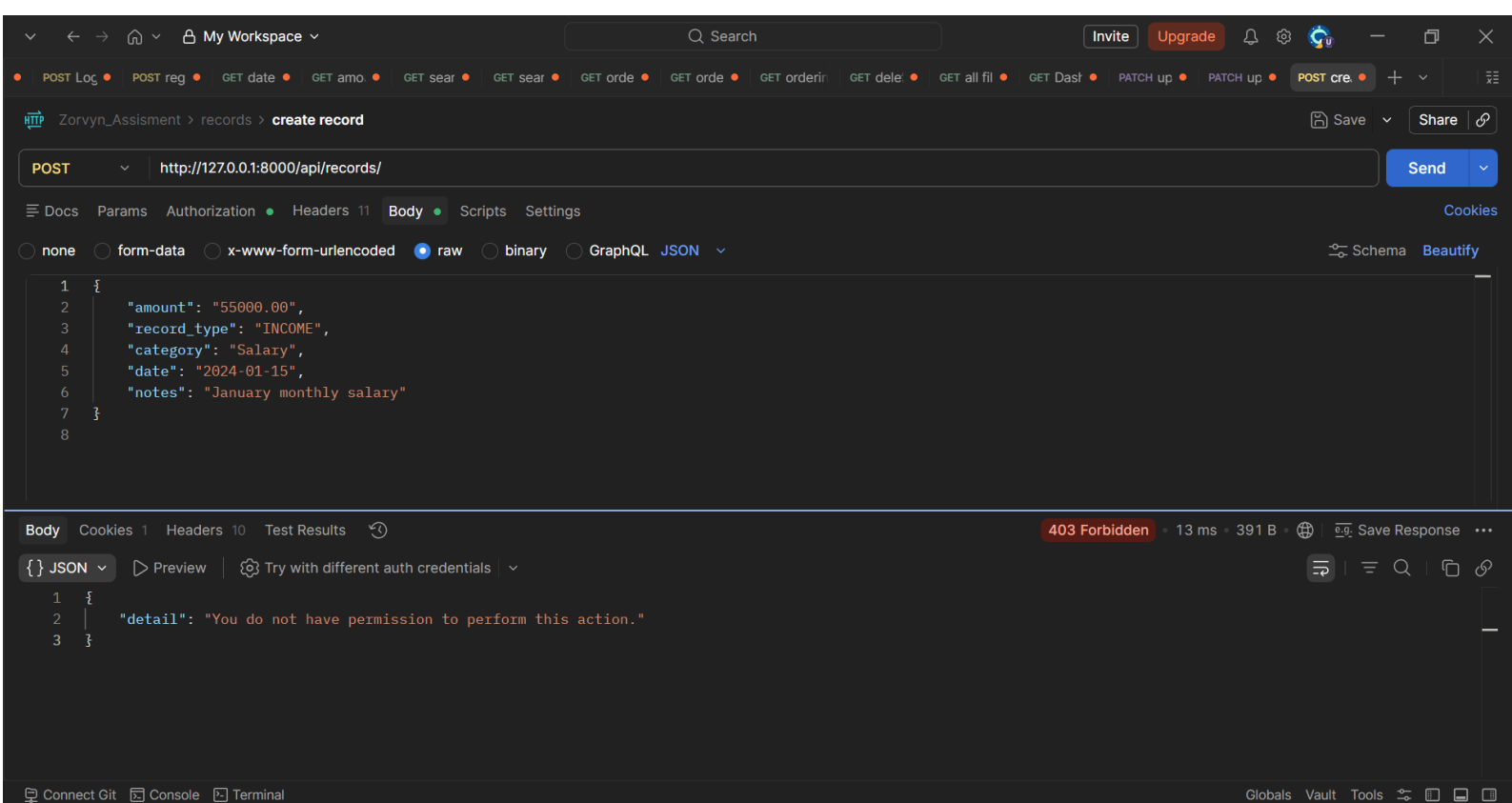


Figure 8: Create Records – 403 Forbidden (Non-Admin)

API 5: View All Records

Field	Details
Method	GET
URL	/api/records/
Permission	Analyst, Admin
Auth Required	Yes

Description: Returns all active financial records. Supports filtering, searching, and ordering.

Filtering Options:

- ?record_type=EXPENSE
- ?record_type=INCOME
- ?date_from=2024-01-01&date_to=2024-01-31
- ?min_amount=5000&max_amount=20000
- ?search=rent
- ?ordering=-amount
- ?ordering=date

?show_deleted=true (Analyst and Admin only)

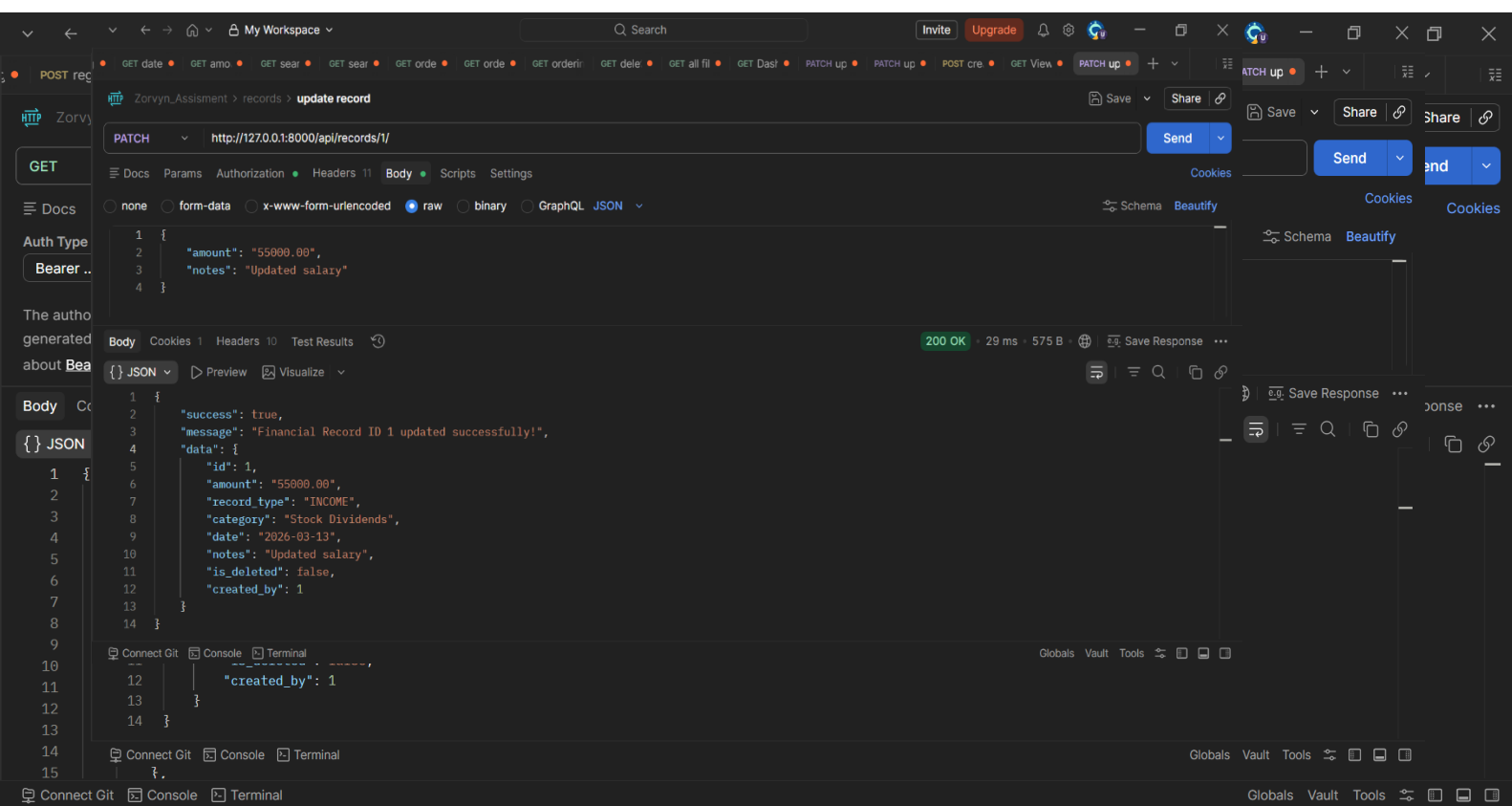


Figure 9: View All Records – 200 OK

API 6: Update Records

Field	Details
Method	PATCH
URL	/api/records/{id}/
Permission	Admin
Auth Required	Yes

Description: Allow Admin to update financial records.

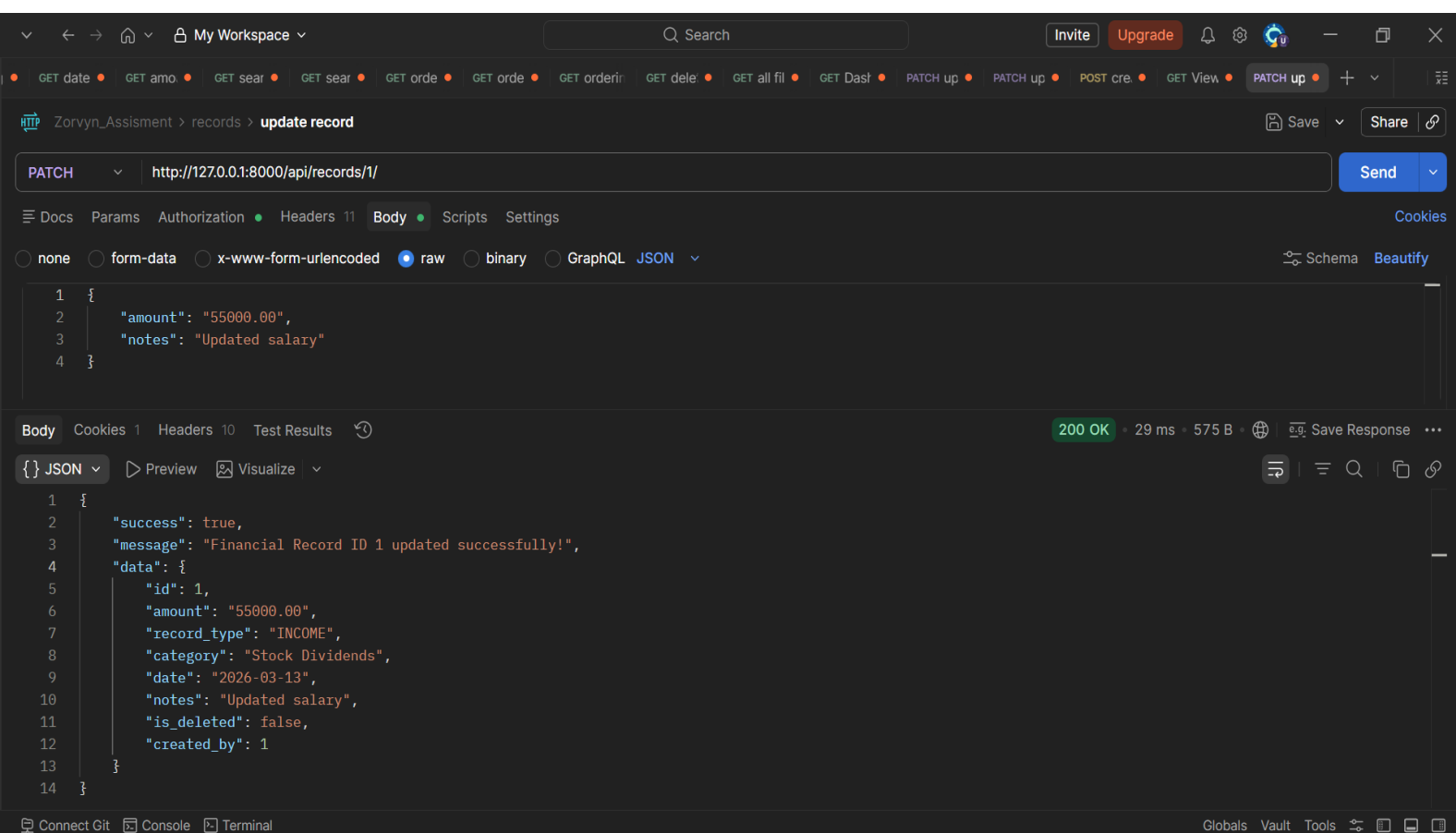


Figure 10: Update Records – (200 OK)

API 7: Delete Records

Field	Details
Method	DELETE
URL	/api/records/id/
Permission	Admin
Auth Required	Yes

Description: Records are not permanently deleted from the database. Instead, the `is_deleted` flag is set to `True`. This preserves financial audit trails. Deleted records can be viewed using `?show_deleted=true` by Analyst or Admin.

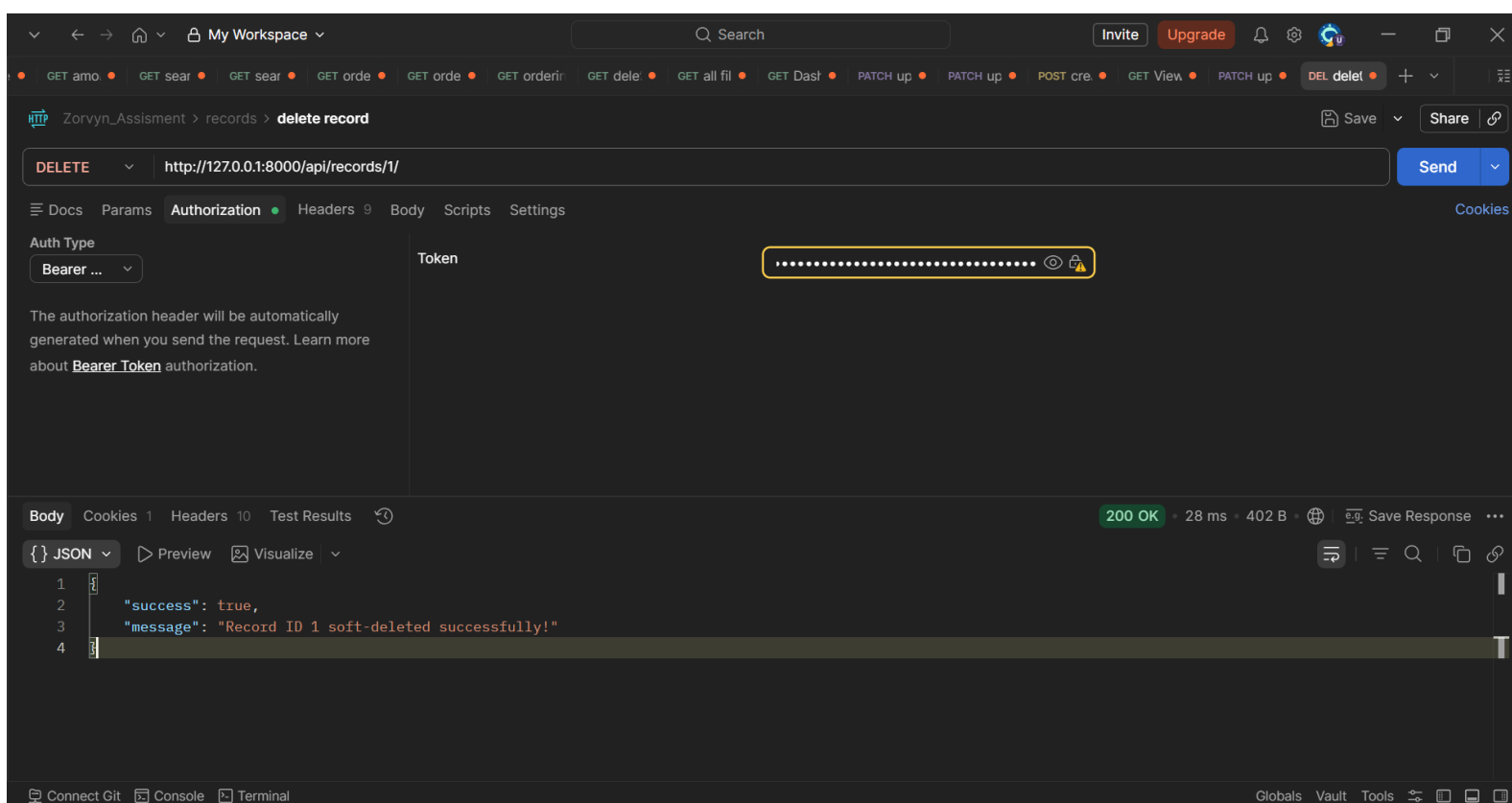


Figure 11: Delete Record (200 OK)

API 8: Filter Records

Field	Details
Method	GET
URL	/api/records ?date=2024-01-15
Permission	Analyst, Admin
Auth Required	Yes

Description: Records can be filtered by date, type, category and also supports ?date_from, ?date_to, ?min_amount, ?max_amount.

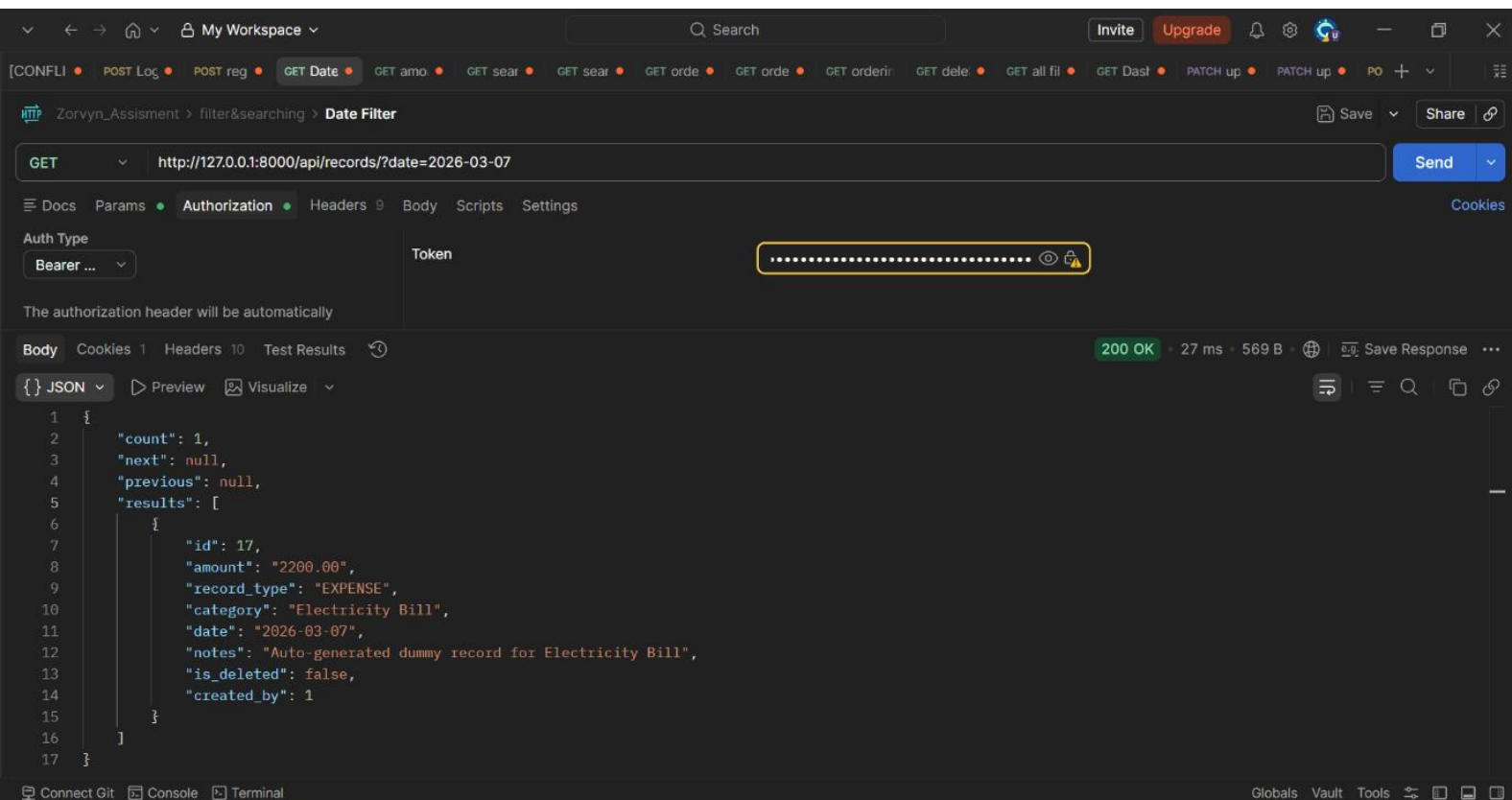


Figure 12: Filter By Date (200 OK)

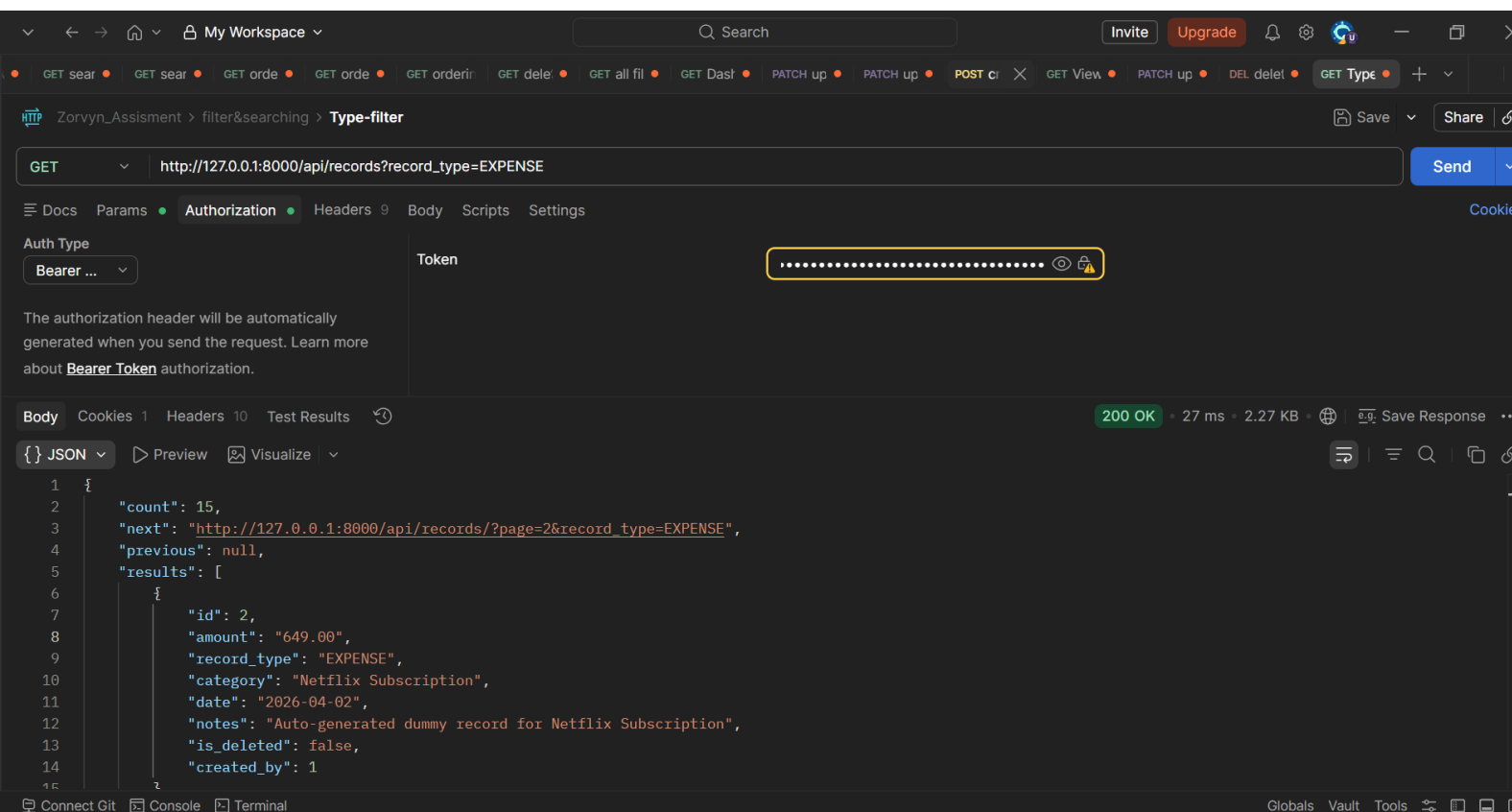


Figure 13: Filter By Type (200 OK)

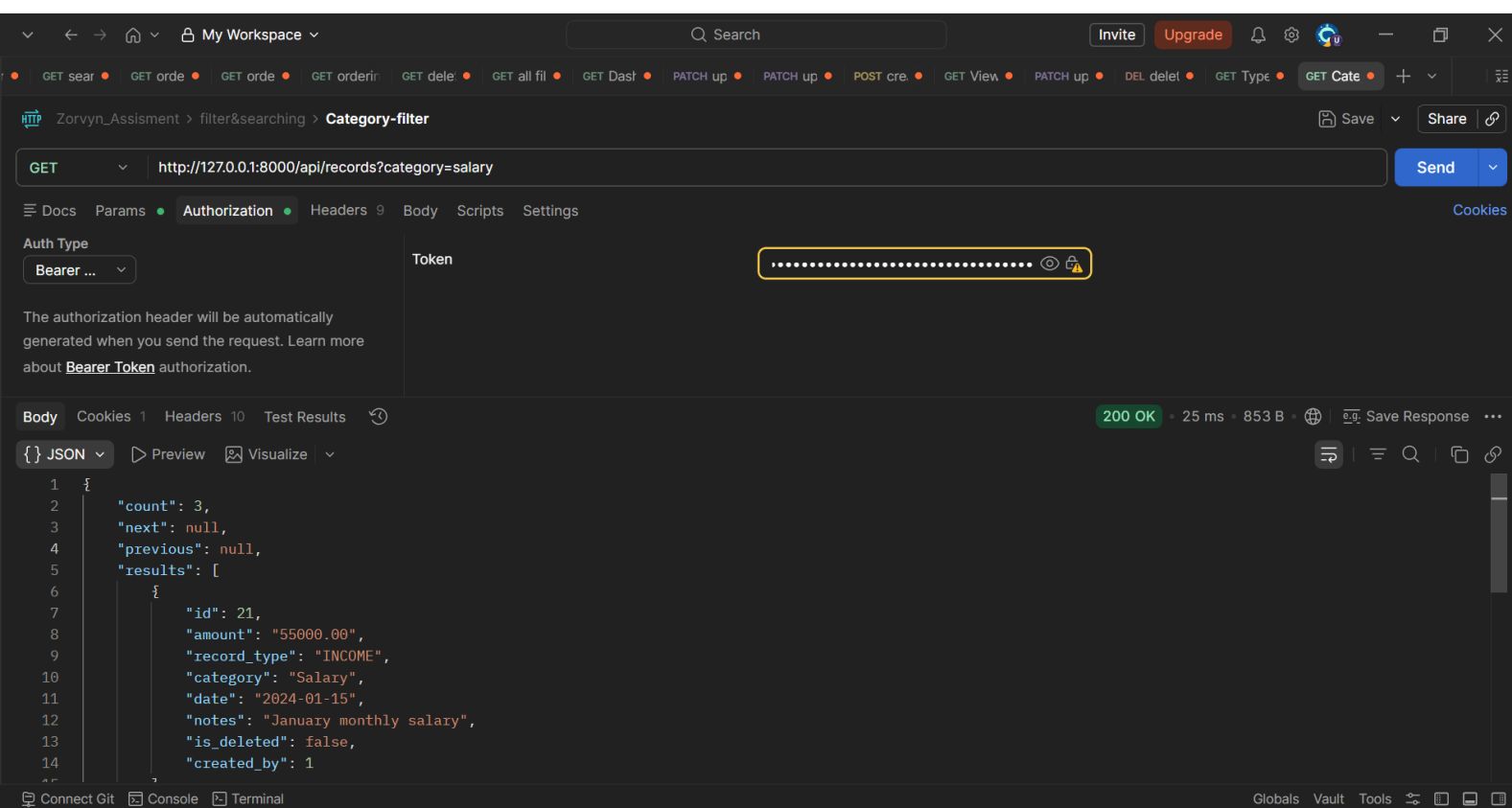


Figure 14: Filter By Category

API 9: Dashboard Summary

Field	Details
Method	GET
URL	/api/dashboard/summary/
Permission	Viewer, Analyst, Admin
Auth Required	Yes

Description: Returns aggregated financial data including total income, total expenses, net balance, category-wise expense breakdown, monthly trends, and recent activity. Only active records are included in calculations.

Success Response — 200 OK:

```
{
  "success": true,
  "message": "Dashboard analytics fetched successfully!",
  "data": {
    "total_income": 278500.0,
    "total_expense": 68047.0,
```

```
"net_balance": 210453.0,
"expense_by_category": [
  {
    "category": "House Rent",
    "total": 36000.0
  }
],
"monthly_trends": [
  {
    "month": "2024-01-01",
    "record_type": "INCOME",
    "total": 165000.0
  },
],
"recent_activity": [
  {
    "id": 14,
    "record_type": "INCOME",
    "amount": 4500.0,
    "category": "Stock Dividends",
    "date": "2026-04-04",
    "created_by__username": "admin"
  },
],
}
```

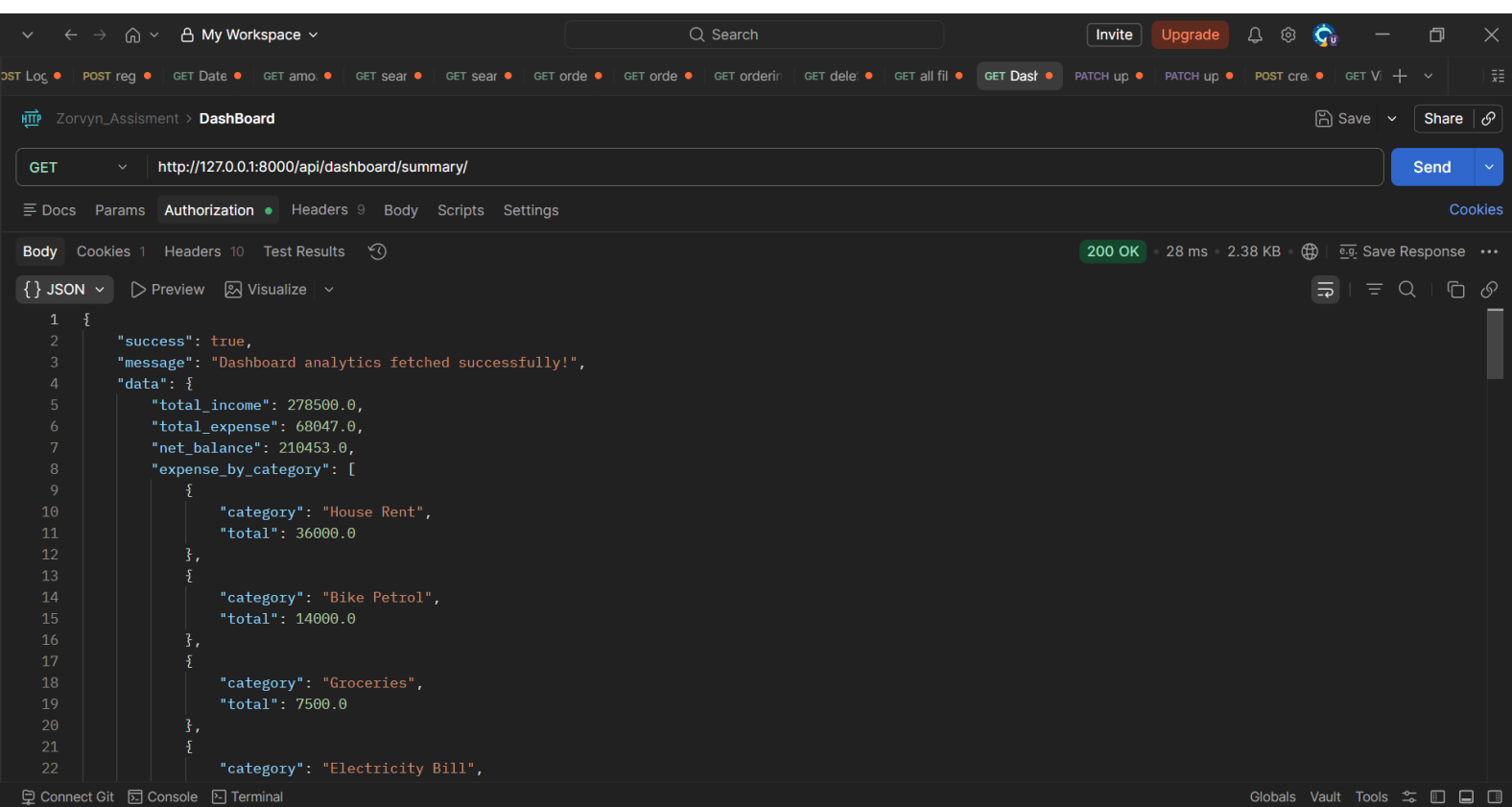


Figure 15: Dashboard Summary - 200 OK

API 10: Search Records

Field	Details
Method	GET
URL	/api/records/
Permission	Analyst, Admin
Auth Required	Yes

Description: “Records can be searched by category and notes fields using the search parameter. Results can be ordered by date or amount.”

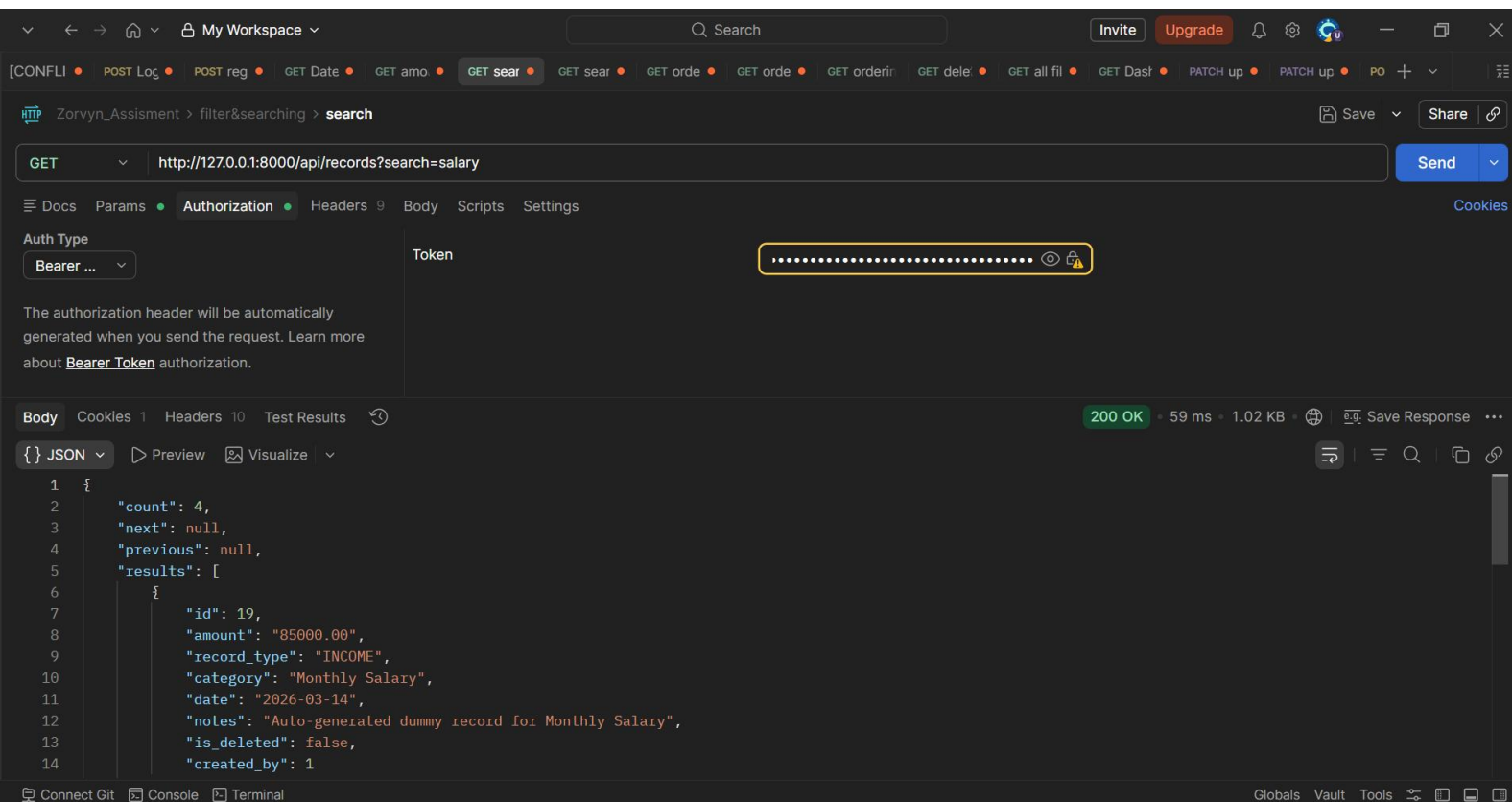


Figure 16: Search Record - 200 OK

Database Setup:

This project uses MySQL as the database. The database name is `finance_db`. All financial records, users, and role information are stored in MySQL tables which are automatically created by Django migrations.

The following screenshot shows the `finance_db` database successfully created in MySQL Workbench with all required tables generated after running migrations.

MySQL Workbench

MySQL x

File Edit View Query Database Server Tools Scripting Help

Query 1 x SQL File 3* SQL File 4*

Limit to 1000 rows

```

2 • use finance_db;
3 • show tables;
4
5 • select * from users_user;
6
7

```

Result Grid

	id	password	last_login	is_superuser	username	first_name	last_name	email	is_staff	is_active	date_joined	role
▶	1	pbkdf2_sha256\$1000000\$G5U74CQhlc1F6YmU...	2026-04-05 17:02:49.876328	1	admin			admin@gmail.com	1	1	2026-04-05 06:45:39.000000	ADM
	2	pbkdf2_sha256\$1000000\$fg1GE6QeS8TCwXs...	HIDE	1	saqb	mohd	saqb	saqb@gmail.com	1	1	2026-04-05 06:46:41.000000	ADM
	3	pbkdf2_sha256\$1000000\$saUyu7e4NmedrYDjA...	HIDE	0	ansari			ansari@gmail.com	0	1	2026-04-05 06:47:45.000000	AN
	4	pbkdf2_sha256\$1000000\$ENidD7hIpuK4v0Zq0...	HIDE	0	mohammad			mohammad@gmail.com	0	1	2026-04-05 06:48:30.000000	VIE
	5	pbkdf2_sha256\$1000000\$bRbqWx5JD9P1P4C...	HIDE	0	abdullah			abdullah@gmail.com	0	1	2026-04-05 16:54:09.192509	VIE
	6	pbkdf2_sha256\$1000000\$Xka7u83IOBmPRM2L...	HIDE	0	kashif			kashif@gmail.com	0	1	2026-04-05 17:06:00.585488	AN
		HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE

users_user 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	01:13:29	use finance_db	0 row(s) affected	0.000 sec
✓ 2	01:13:31	show tables	11 row(s) returned	0.000 sec / 0.000 sec
✓ 3	01:13:48	select * from users_user LIMIT 0, 1000	6 row(s) returned	0.016 sec / 0.000 sec

Figure 17: User Table

MySQL Workbench

MySQL x

File Edit View Query Database Server Tools Scripting Help

Query 1 x SQL File 3* SQL File 4*

Limit to 1000 rows

```

2 • use finance_db;
3 • show tables;
4
5 • select * from users_user;
6 • select * from records_financialrecord;
7

```

Result Grid

	id	amount	record_type	category	date	notes	is_deleted	created_by_id
▶	1	55000.00	INCOME	Stock Dividends	2026-03-13	Updated salary	1	1
	2	649.00	EXPENSE	Netflix Subscription	2026-04-02	Auto-generated dummy record for Netflix Subsc...	0	1
	3	3500.00	EXPENSE	Bike Petrol	2026-03-27	Auto-generated dummy record for Bike Petrol	0	1
	4	3500.00	EXPENSE	Bike Petrol	2026-03-11	Auto-generated dummy record for Bike Petrol	0	1
	5	649.00	EXPENSE	Netflix Subscription	2026-03-29	Auto-generated dummy record for Netflix Subsc...	0	1
	6	7500.00	EXPENSE	Groceries	2026-04-03	Auto-generated dummy record for Groceries	0	1
	7	12000.00	INCOME	Diwali Bonus	2026-03-30	Auto-generated dummy record for Diwali Bonus	0	1
	8	3500.00	EXPENSE	Bike Petrol	2026-03-21	Auto-generated dummy record for Bike Petrol	0	1

records_financialrecord 4 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 2	01:13:31	show tables	11 row(s) returned	0.000 sec / 0.000 sec
✓ 3	01:13:48	select * from users_user LIMIT 0, 1000	6 row(s) returned	0.016 sec / 0.000 sec
✓ 4	01:16:11	show tables	11 row(s) returned	0.000 sec / 0.000 sec
✓ 5	01:16:30	select * from records_financialrecord LIMIT 0, 1000	23 row(s) returned	0.000 sec / 0.000 sec

Figure 18: Finance record Table

Django Admin Panel:

Description: "The Django Admin panel provides a summary view of total income, expenses, and net balance along with full record management."

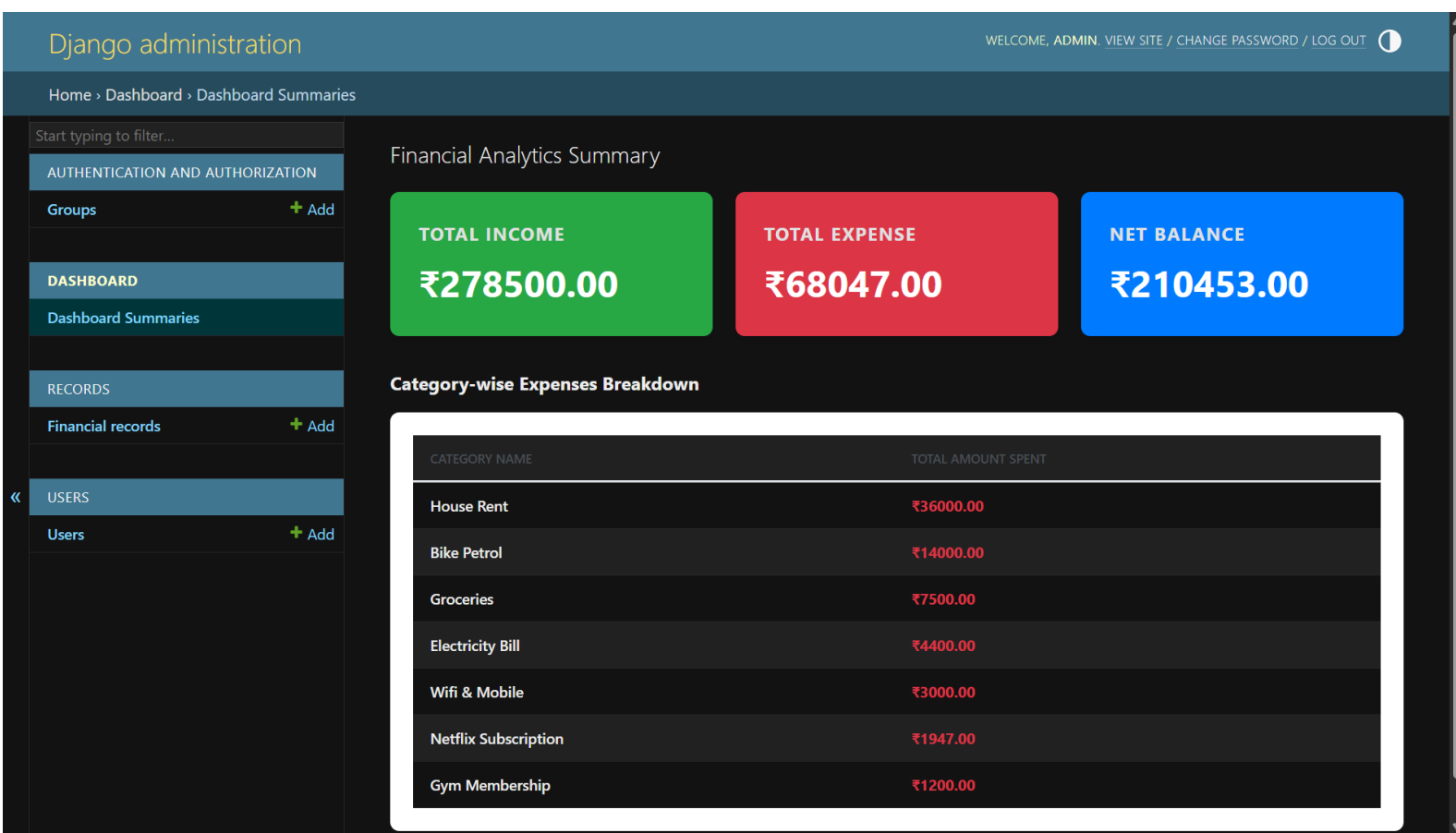


Figure 19: Django Admin – Summary

Pagination

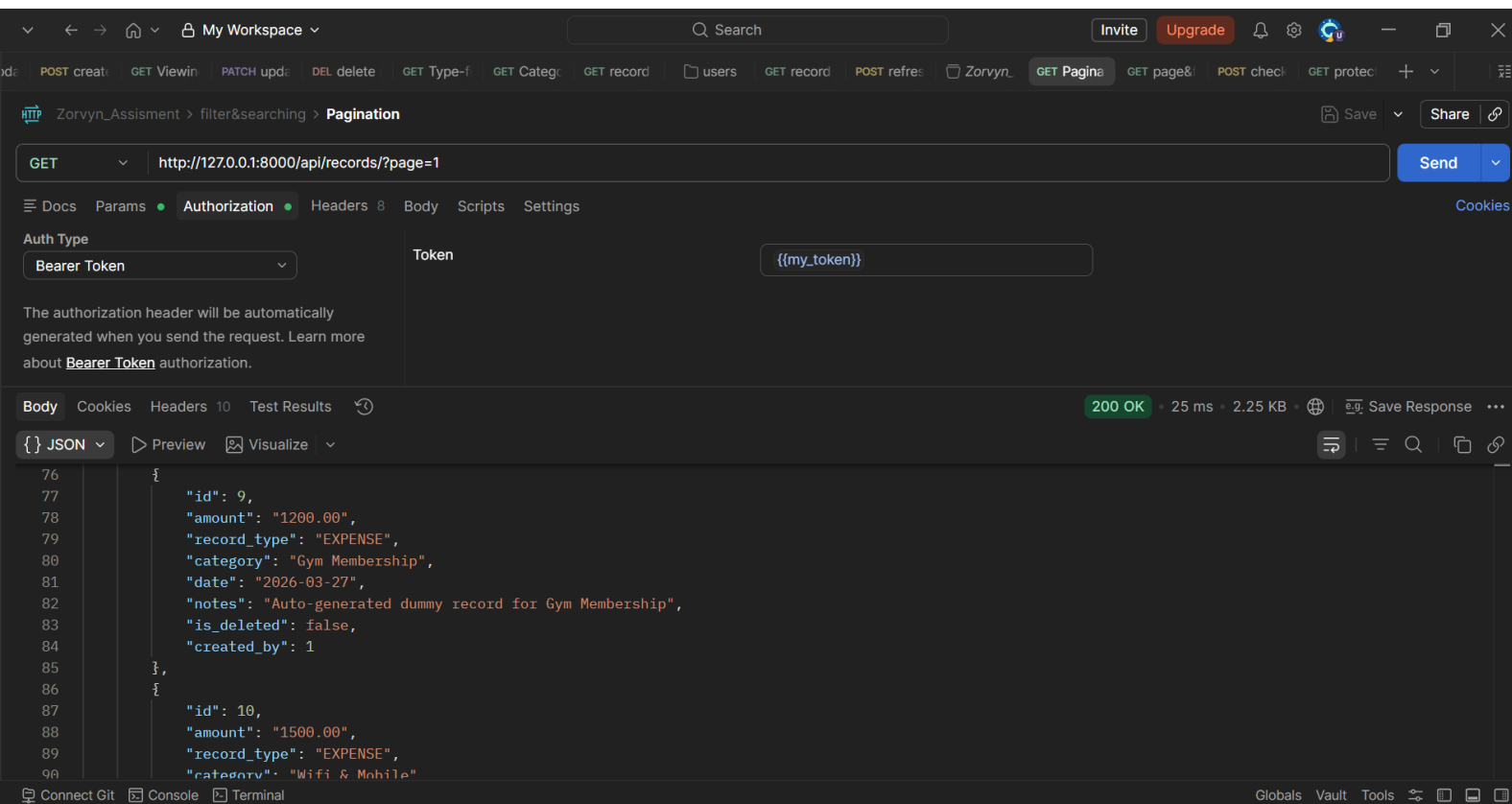


Figure 20: Pagination 200 OK

Rate Limiting

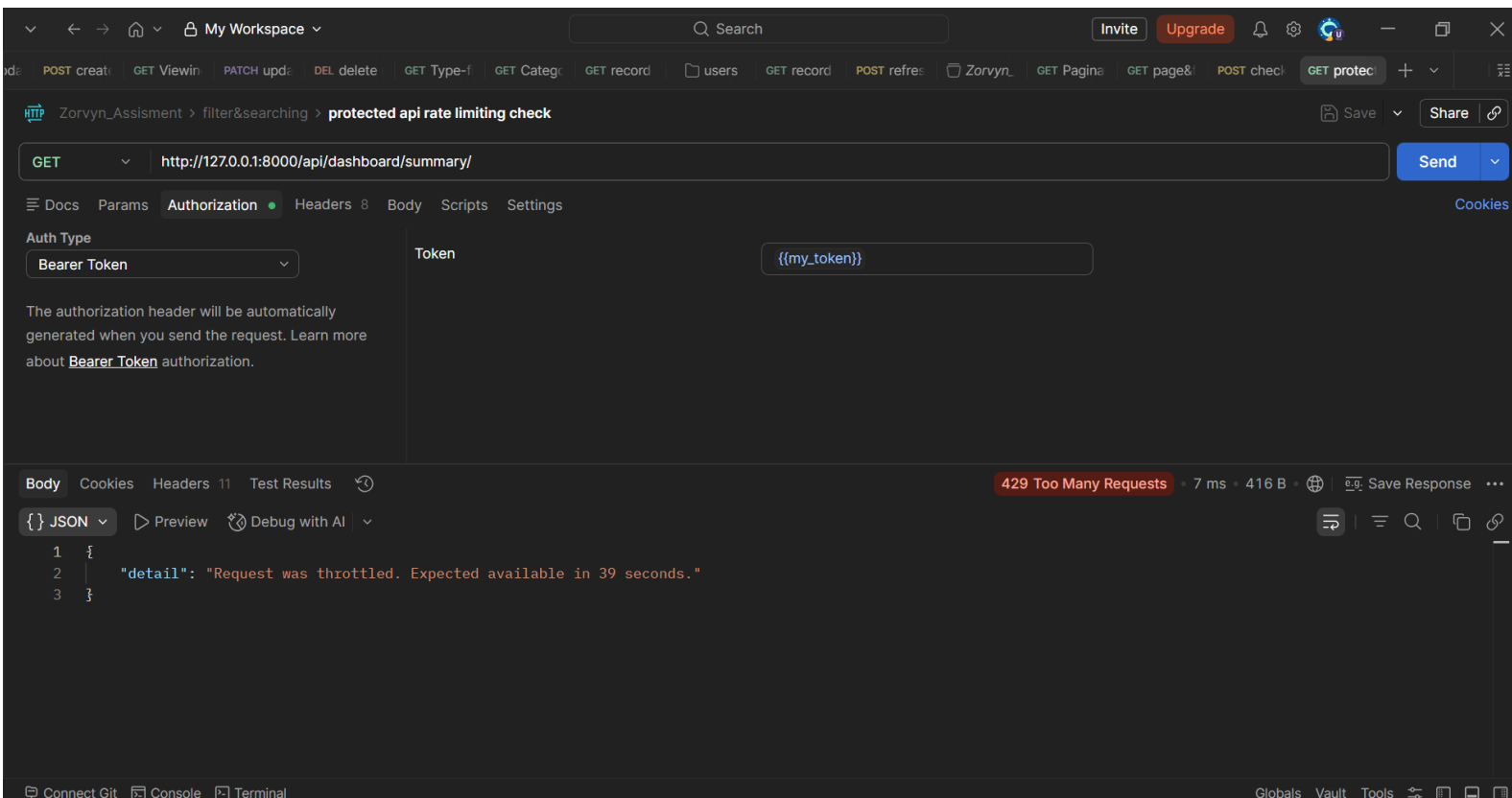


Figure 21: Rate Limiting – Too Many Requests (429)

Implemented Enhancements:

- JWT Authentication
- Unique email validation
- Soft delete functionality
- Pagination for record listing
- Search support
- Django Admin with summary view
- Rate Limiting
- MySQL database
- Swagger UI (<http://127.0.0.1:8000/swagger/api/docs>)
- API documentation

Key Design Decisions

1. Soft Delete Records are never permanently deleted. This preserves financial history and audit trails which is critical in finance systems.

2. Role Based Access Control Each API is protected by custom permission classes. Viewers can only see dashboard summary, Analysts can view records, and Admins have full control.

3. Unique Email Validation Both username and email are enforced as unique to prevent duplicate accounts and security risks like unauthorized password resets.

4. SET_NULL on User Delete If a user is deleted, their financial records remain intact. Only the `created_by` field becomes null, preserving all financial data.

5. DB Level Aggregations Dashboard calculations use Django ORM aggregations instead of Python loops, making the system scalable for large datasets.

6. Stateless Authentication (JWT) Instead of traditional session-based auth, JSON Web Tokens (JWT) are used. This makes the backend fully stateless and scalable, as the server does not need to store active sessions in the database.

7. API Throttling (Rate Limiting) To protect the backend from DDoS attacks and brute-force attempts, Rate Limiting has been configured. This ensures system stability by restricting the number of requests a single user can make within a timeframe.